Credit Risk in Germany
Technical Companion

How the analysis was built: data, code and design choices, step by step

Ettore Mancuso
ettoremancuso6@gmail.com
October 2026

Why this document

The paper *Credit Risk in Germany: From System-Wide Signals to a Borrower-Level PD Model* reports results. This companion shows how I got them. It follows the same order as the analysis, and under every chart, table and code excerpt there is a Code: line that points to the exact file, notebook cell or function that produced it, so anyone can open the repository and find it in a few seconds.

I wrote it mainly for people who want to judge the programming side of the project. In short: the analysis is a single Python notebook that runs from start to finish in well under a minute, the reusable logic sits in two small modules, SQL handles the segment analysis, an R script cross-checks the main model, and every output is exported as CSV for a Power BI report. Notebook cells are cited by their execution number, `[n]`, as shown in Jupyter.

Table 1: Tools used and what each one does in the project.

Tool	Libraries / setup	Role
Python 3	pandas, NumPy, SciPy	Data handling, numerical root finding (`brentq`), normal distribution for the capital formula
	scikit-learn	Train/test split, cross-validation, random forest, gradient boosting, ROC/AUC
	statsmodels	Unpenalised logit with standard errors and p-values
	matplotlib	All six figures (exported as vector PDF and PNG)
SQL	SQLite (in memory)	Default rates by segment, queries kept in a separate `.sql` file
R	base R (`glm`)	Independent cross-check of the logistic model on the same split
Jupyter	nbconvert	Main analysis notebook, also executable from the command line
	pytest, GitHub Actions	Unit tests for the two modules, run automatically on every push
Power BI	CSV exports	Reporting layer built on `outputs/`
LaTeX	XeLaTeX	Paper and this companion
Git / GitHub		Version control and publication

1. Architecture of the repository

Figure 1 shows how data move through the project. Raw inputs never change; everything downstream can be deleted and rebuilt by running the notebook.

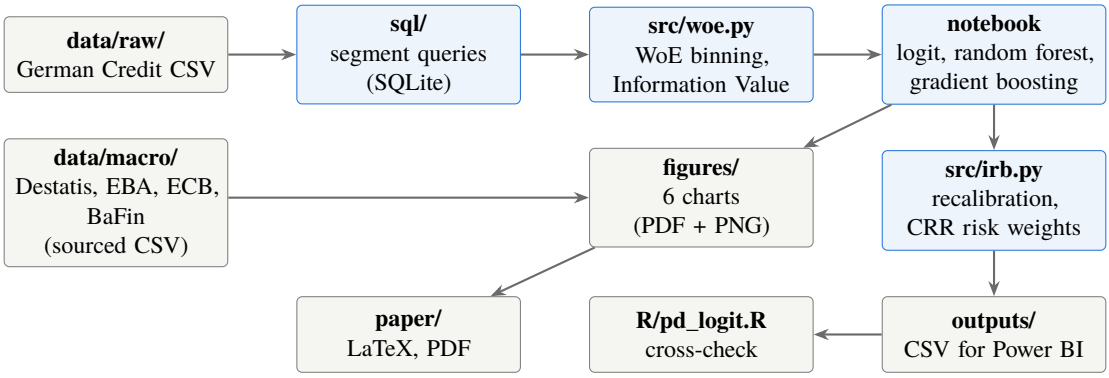


Figure 1: Data flow through the repository. Blue boxes contain the analytical code.

Code: `README.md (repository map)`

Three choices shaped the structure:

- **Logic in modules, story in the notebook.** Functions that I might reuse (WoE encoding, the capital formula) live in `src/` and are imported by the notebook. The notebook only calls them and shows results.
- **One source per number.** Every hand-collected value has a `source` column next to it, so a reader can check any figure in the paper against its origin.
- **Reproducible by default.** A fixed random seed (`SEED = 42`), relative paths and a list of required packages. I tested this by copying the repository to an empty folder, deleting all outputs and running everything again: the results were identical.

2. Step 1: collecting and checking system-level data

Public banking statistics are scattered across institutions and change definition over time, so I collected the values by hand into three small CSV files. Each row records where the number comes from. The notebook loads them with pandas and computes the derived figures quoted in the paper, such as the change in insolvencies from the 2021 trough.

```
ins = pd.read_csv(ROOT / "data/macro/insolvencies_de.csv")
npl = pd.read_csv(ROOT / "data/macro/npl_indicators.csv", parse_dates=["date"])
ins["yoy_pct"] = ins.corporate_insolvencies.pct_change() * 100
low = ins.loc[ins.corporate_insolvencies.idxmin()]
last = ins.corporate_insolvencies.iloc[-1]
print(f"2025 vs trough: {last / low.corporate_insolvencies - 1:+.1%}")
```

Code: `notebooks/german_credit_risk.ipynb, cell [2] (Python, pandas)`

A simple consistency check caught a gap early: Destatis's historical table stops in 2022, and the 2023 and 2024 values come from press releases. I verified that the reported growth rates (+22.1% and +22.4%) reproduce the levels before accepting them.

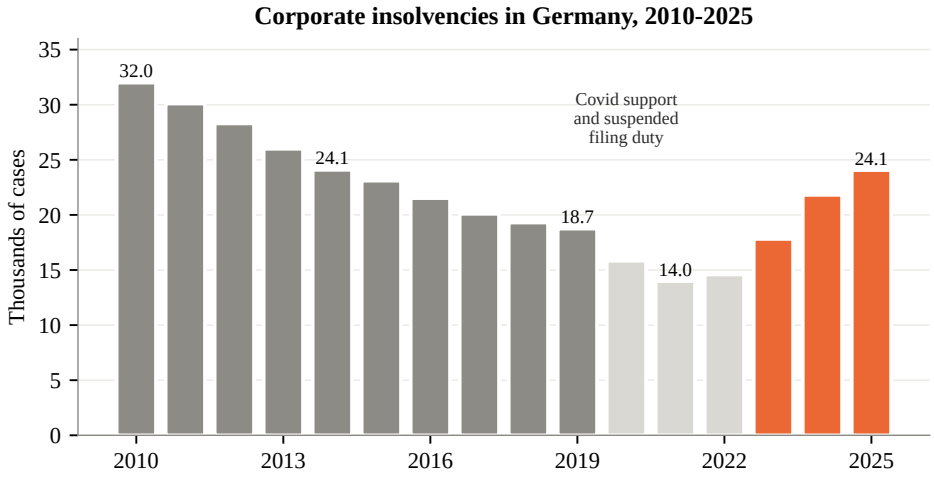


Figure 2: Corporate insolvencies in Germany, 2010–2025 (paper, Figure 1).

Code: `notebooks/german_credit_risk.ipynb, cell [3] (Python, matplotlib) | data: data/macro/insolvencies_de.csv`

For the EU comparison I used a dumbbell chart: one row per indicator, with Germany and the EU joined by a line, so the size and the direction of the gap are visible at once.

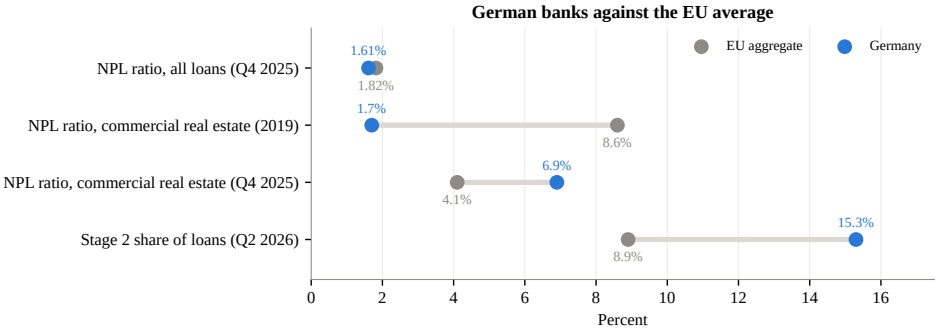


Figure 3: German banks against the EU average (paper, Figure 2).

Code: `notebooks/german_credit_risk.ipynb, cell [4] (Python, matplotlib) | data: data/macro/npl_indicators.csv`

3. Step 2: exploring the loan data with SQL

Before modelling I wanted a feel for the data, and the quickest way to ask "what is the default rate by group?" is SQL. The notebook loads the CSV into an in-memory SQLite database. The queries live in their own file, so they can be read, reused or run in any SQL client without opening Python.

```sql
SELECT CASE
         WHEN duration_in_month <= 12 THEN '01-12 months'
         WHEN duration_in_month <= 24 THEN '13-24 months'
         WHEN duration_in_month <= 36 THEN '25-36 months'
         ELSE '37+ months'
       END                          AS segment,
       COUNT(*)                     AS n,
       SUM(bad)                     AS defaults,
       ROUND(100.0 * AVG(bad), 1)   AS default_rate_pct,
       ROUND(AVG(credit_amount), 0) AS avg_amount_dm
FROM loans
GROUP BY segment
ORDER BY segment;
```

Code: `sql/segments.sql, query by_duration_band (SQL, SQLite)`

The notebook splits the file on its `-- name:` markers and runs each query with `pd.read_sql`:

```python
con = sqlite3.connect(":memory:")
raw.to_sql("loans", con, index=False)
for block in (ROOT / "sql/segments.sql").read_text().split("-- name: ")[1:]:
    name, sql = block.split("\n", 1)
    queries[name.strip()] = sql.strip()
seg = {name: pd.read_sql(q, con) for name, q in queries.items()}
```

Code: `notebooks/german_credit_risk.ipynb, cell [6] (Python + SQL)`

Table 2: Output of the duration query.

Segment	Loans	Defaults	Default rate	Avg. amount (DM)
01–12 months	359	76	21.2%	1,811
13–24 months	411	122	29.7%	2,924
25–36 months	143	57	39.9%	5,274
37+ months	87	45	51.7%	7,646

Code: `sql/segments.sql, executed in notebook cell [6]`

4. Step 3: feature engineering with Weight of Evidence

A credit scorecard does not use raw variables. Each one is split into bins, and each bin is replaced by its Weight of Evidence (WoE), the log ratio between the share of good and bad loans that fall in it. This handles categorical and numeric variables in the same way, makes the relation with default monotone, and gives the Information Value (IV), a measure of how much each variable can separate good from bad loans.

I wrote the encoder myself instead of using a package, partly to understand it and partly to control one detail: the bins must be learned on training data only. The class follows the scikit-learn `fit`/`transform` pattern.

```python
def woe_table(x: pd.Series, y: pd.Series, n_bins: int = 5) -> pd.DataFrame:
    """Bin-level WoE/IV table for a single risk driver.

    Continuous drivers (amount, duration, age...) are cut into quantile bins;
    categorical ones keep their own categories.
    """
    if _is_continuous(x, n_bins):
        x = pd.qcut(x, q=n_bins, duplicates="drop").astype(str)

    tab = (pd.DataFrame({"bin": x.astype(str), "bad": y.to_numpy()})
             .groupby("bin", observed=True)["bad"]
             .agg(n="count", bad="sum"))
    tab["good"] = tab["n"] - tab["bad"]

    k = len(tab)
    tab["pct_good"] = (tab["good"] + SMOOTHING) / (tab["good"].sum() + SMOOTHING * k)
    tab["pct_bad"] = (tab["bad"] + SMOOTHING) / (tab["bad"].sum() + SMOOTHING * k)
    tab["bad_rate"] = tab["bad"] / tab["n"]
    tab["woe"] = np.log(tab["pct_good"] / tab["pct_bad"])
    tab["iv"] = (tab["pct_good"] - tab["pct_bad"]) * tab["woe"]
    return tab.reset_index()
```

Code: `src/woe.py`, function `woe_table()` (Python, pandas)

```python
    def fit(self, X: pd.DataFrame, y: pd.Series) -> "WoEEncoder":
        for col in X.columns:
            x = X[col]
            if _is_continuous(x, self.n_bins):
                _, edges = pd.qcut(x, q=self.n_bins, retbins=True, duplicates="drop")
                # open-ended outer bins: a test value outside the training range
                # still lands in a bin instead of becoming NaN
                edges[0], edges[-1] = -np.inf, np.inf
                self.edges_[col] = edges
            tab = woe_table(self._to_bins(col, x), y, self.n_bins)
            self.mappings_[col] = dict(zip(tab["bin"], tab["woe"]))
        return self
```

Code: `src/woe.py`, method `WoEEncoder.fit()`

Two details are worth pointing out. The smoothing constant of 0.5 (lines 43–44) avoids an infinite WoE when a bin has no defaults. And in `fit`, the outer bin edges are set to $\pm\infty$ (line 76), so a test value outside the training range still falls into a bin instead of producing a missing value.

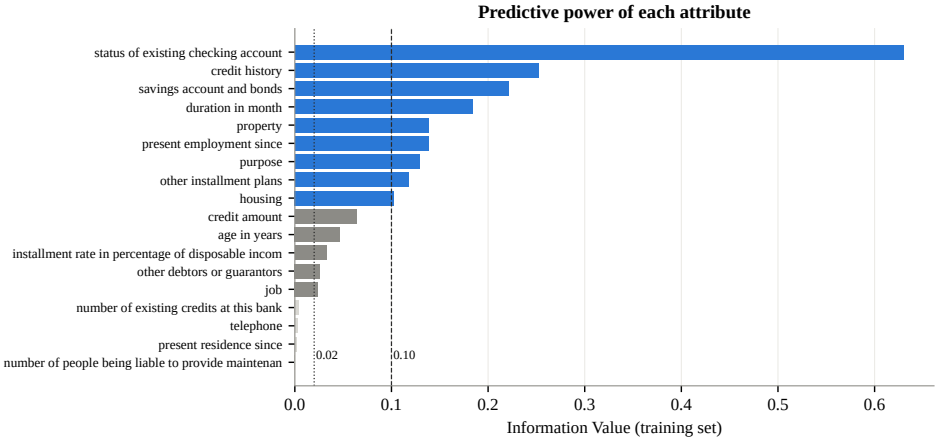


Figure 4: Information Value of each attribute (paper, Figure 3).

Code: `notebooks/german_credit_risk.ipynb`, cells [7] (WoE fit, IV table) and [8] (matplotlib) | `src/woe.py`

5. Step 4: three models and a fair comparison

I compared a logistic scorecard with two machine-learning models. The main programming issue was *data leakage*: if the WoE bins were fitted on the whole training set before cross-validation, every validation fold would have "seen" its own outcomes through the encoding, and the scorecard would look better than it is. To prevent it I wrapped the encoder and the regression in one small class, so that both are refitted inside every fold:

```
class WoELogit:
    """WoE encoding + logistic regression, refitted inside each CV fold."""
    def __init__(self, features): self.features = features
    def fit(self, X, y):
        self.enc = WoEEncoder(5).fit(X[self.features], y)
        self.lr = LogisticRegression(C=1.0, max_iter=1000).fit(
            self.enc.transform(X[self.features]), y)
        return self
    def predict_proba(self, X):
        return self.lr.predict_proba(self.enc.transform(X[self.features]))
```

Code: `notebooks/german_credit_risk.ipynb`, cell [9] (Python, scikit-learn)

The random forest and the gradient boosting model are scikit-learn `Pipelines` with one-hot encoding. All three go through the same loop, so the comparison is like for like:

```
skf = StratifiedKFold(n_splits=5, shuffle=True, random_state=SEED)
for name, model in models.items():
    cv_auc = []
    for tr, va in skf.split(X_tr, y_tr):
        m = model.fit(X_tr.iloc[tr], y_tr.iloc[tr])
        cv_auc.append(discrimination(y_tr.iloc[va],
                                     m.predict_proba(X_tr.iloc[va])[:, 1])["AUC"])
    model.fit(X_tr, y_tr)                          # refit on the full training set
    p = model.predict_proba(X_te)[:, 1]            # score the untouched test set
```

Code: `notebooks/german_credit_risk.ipynb`, cell [9]

Table 3: Model comparison (paper, Table 2).

Model	CV AUC	CV s.d.	Test AUC	Gini	KS	Brier
Logistic scorecard (WoE)	0.758	0.019	0.801	0.601	0.502	0.159
Random forest	0.783	0.035	0.794	0.587	0.481	0.166
Gradient boosting	0.764	0.019	0.796	0.592	0.462	0.159

Code: `notebooks/german_credit_risk.ipynb, cell [9] | exported to outputs/model_comparison.csv`

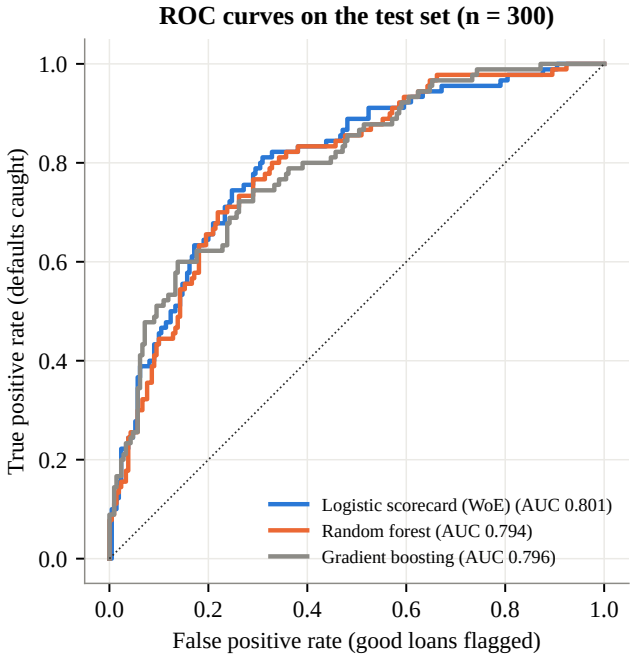


Figure 5: ROC curves on the test set (paper, Figure 4).

Code: `notebooks/german_credit_risk.ipynb, cell [10] (scikit-learn roc_curve, matplotlib)`

6. Step 5: checking the model from a second angle, in R

Two checks make me more confident that the scorecard is sound. In Python, statsmodels gives the unpenalised logit with standard errors: 13 of 14 coefficients have the expected sign (cell [11]). In R, I fitted a plain logistic regression on dummy-coded categories. To make the comparison meaningful, the R script reads the exact test rows that Python used, which the notebook saves to `outputs/test_index.csv`. Python indexes from 0 and R from 1, hence the `+ 1` on line 12.

```r
loans <- read.csv("data/raw/germancredit.csv", stringsAsFactors = TRUE)
loans$bad <- as.integer(loans$creditability == "bad")
loans$creditability <- NULL
loans$personal_status_and_sex <- NULL   # excluded for fairness, as in the notebook
loans$foreign_worker <- NULL

test_rows <- read.csv("outputs/test_index.csv")$row + 1   # Python index is 0-based
train <- loans[-test_rows, ]
test  <- loans[test_rows, ]

fit <- glm(bad ~ ., data = train, family = binomial(link = "logit"))
cat(sprintf("Training observations: %d | AIC: %.1f\n", nrow(train), AIC(fit)))

# AUC via the Mann-Whitney statistic (no packages needed)
auc <- function(y, p) {
  r <- rank(p)
  n1 <- sum(y == 1); n0 <- sum(y == 0)
  (sum(r[y == 1]) - n1 * (n1 + 1) / 2) / (n1 * n0)
}
p_test <- predict(fit, newdata = test, type = "response")
a <- auc(test$bad, p_test)
cat(sprintf("Test AUC: %.3f | Gini: %.3f\n", a, 2 * a - 1))
```

Code: R/pd_logit.R (base R, glm) | output: Test AUC 0.792, Gini 0.584

The R model reaches a test AUC of 0.792, against 0.801 for the scorecard: a different encoding and a different language, but almost the same answer. I computed the AUC with the Mann–Whitney formula so that the script needs no extra packages.

7. Step 6: from scores to rating grades

To place every loan in a grade without in-sample bias, each one is scored by a model trained on the other four fifths of the data (out-of-fold prediction). The PDs are then moved to a realistic 3% average. The function shifts the log-odds of every loan by the same factor and finds that factor numerically with SciPy's `brentq`, so the ranking, and therefore the AUC, does not change.

```python
def recalibrate(pd_, target_rate: float) -> np.ndarray:
    """Move the average PD of a sample to `target_rate` (calibration to central tendency).

    The German Credit sample over-represents defaults (30% bad), so raw PDs are
    far too high for capital purposes. Multiplying every borrower's odds by the
    same constant c is an intercept shift in the logit: ranking and AUC stay
    the same, only the level moves. c is solved numerically with Brent's method.
    """
    pd_ = np.clip(np.asarray(pd_, dtype=float), 1e-6, 1 - 1e-6)
    odds = pd_ / (1 - pd_)

    def gap(c: float) -> float:
        return np.mean(odds * c / (1 + odds * c)) - target_rate

    c = brentq(gap, 1e-8, 1e4)
    return odds * c / (1 + odds * c)
```

Code: src/irb.py, function recalibrate() (Python, NumPy, SciPy)

```python
oof = np.zeros(len(X))
for tr, va in skf.split(X, y):   # each loan is scored by a model that never saw it
    model = WoELogit(SELECTED).fit(X.iloc[tr], y.iloc[tr])
    oof[va] = model.predict_proba(X.iloc[va])[:, 1]
port["pd"] = np.maximum(recalibrate(oof, CT), PD_FLOOR_RETAIL)
port["grade"] = pd.cut(port.pd, edges, labels=labels, right=False)
```

Code: `notebooks/german_credit_risk.ipynb, cell [12] (Python, pandas, SciPy binomtest)`

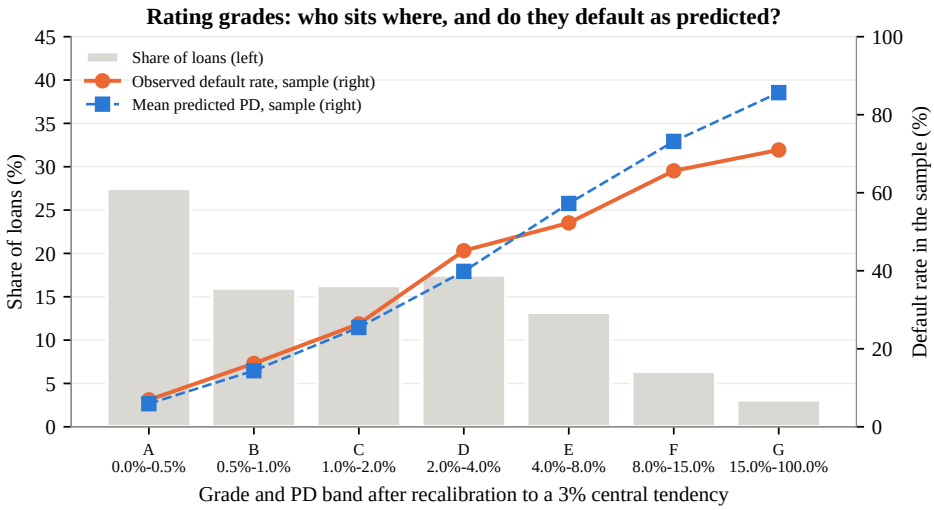


Figure 6: Rating grades: share of loans, observed and predicted default rates (paper, Figure 5).

Code: `notebooks/german_credit_risk.ipynb, cell [13] (matplotlib, twin axes)`

8. Step 7: translating PDs into regulatory capital

The capital formula of the CRR for retail exposures is short enough to write in a few lines, and writing it myself was the best way to understand it. `norm.ppf` and `norm.cdf` are the inverse and the cumulative standard normal distribution from SciPy; the PD floor of 0.05% introduced by CRR3 is applied first.

```python
def other_retail_correlation(pd_: np.ndarray) -> np.ndarray:
    """Asset correlation: 16% for the safest borrowers, falling to 3% as PD rises."""
    w = (1 - np.exp(-35 * pd_)) / (1 - np.exp(-35))
    return 0.03 * w + 0.16 * (1 - w)


def capital_k(pd_, lgd: float) -> np.ndarray:
    """Unexpected-loss capital per unit of EAD (downturn loss minus expected loss)."""
    pd_ = np.maximum(np.asarray(pd_, dtype=float), PD_FLOOR_RETAIL)
    rho = other_retail_correlation(pd_)
    stressed_pd = norm.cdf((norm.ppf(pd_) + np.sqrt(rho) * norm.ppf(CONFIDENCE))
                           / np.sqrt(1 - rho))
    return lgd * stressed_pd - lgd * pd_


def risk_weight(pd_, lgd: float) -> np.ndarray:
    """Risk weight as a share of EAD, e.g. 0.75 = 75%."""
    return 12.5 * capital_k(pd_, lgd)
```

Code: `src/irb.py, functions other_retail_correlation(), capital_k(), risk_weight()`

Because the functions are vectorised with NumPy, the same code computes the risk weight of one loan,

of 1,000 loans, or of 300 points along a curve for the chart. The notebook then aggregates risk-weighted assets, expected loss and the output-floor test, and repeats the calculation for six combinations of PD and LGD.

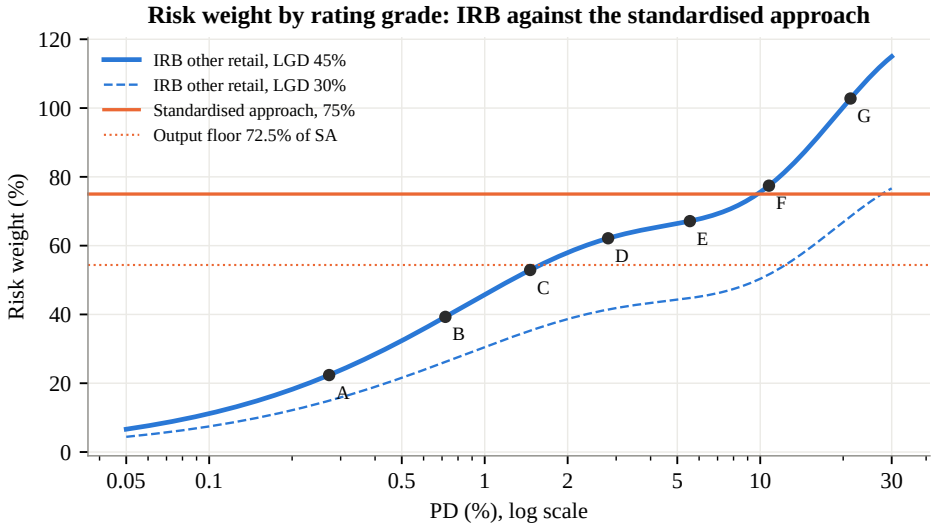


Figure 7: IRB risk weight by grade against the standardised approach (paper, Figure 6).

Code: `notebooks/german_credit_risk.ipynb, cells [14] (capital), [15] (sensitivity), [16] (chart) | src/irb.py`

9. Step 8: reporting and reproducibility

The last cell exports five CSV files to `outputs/`: one row per loan with PD, grade, risk weight and expected loss, plus summaries by grade, the model comparison, the Information Values and the capital sensitivity. They are shaped for a Power BI report: one fact table and four small summary tables that Power BI can read straight from the folder, so the report refreshes whenever the notebook is run again.

```
port.to_csv(OUT / "scored_portfolio.csv", index_label="loan_id")
cols = ["rw_irb", "el_pct", "exposure_share"]
g.join(by_grade[cols]).to_csv(OUT / "grade_summary.csv")
res.to_csv(OUT / "model_comparison.csv")
iv_tab.to_csv(OUT / "information_values.csv")
sens.to_csv(OUT / "capital_sensitivity.csv", index=False)
```

Code: `notebooks/german_credit_risk.ipynb, cell [17]`

The two modules are covered by unit tests in `tests/`. They check the risk weight against a value worked out by hand from the CRR formula, that the PD floor is applied, that recalibration hits its target without changing the ranking, that a pure-noise variable gets an Information Value close to zero, and that categories or values never seen in training do not break the encoder. A GitHub Actions workflow runs them on every push. The whole project can be rebuilt from the command line:

```
pip install -r requirements.txt pytest
pytest -q
jupyter nbconvert --to notebook --execute --inplace notebooks/german_credit_risk.ipynb
Rscript R/pd_logit.R
```

Code: `README.md, section ``How to run''`

10. What I would do differently

The project has clear limits, and some of them are programming choices I would change with more time.

- **More tests.** The modules are tested, the notebook is not. I would move the model-comparison loop into a function and test it on a small synthetic dataset.
- **Hyperparameter tuning.** The random forest and the gradient boosting model use reasonable defaults. A grid search inside the cross-validation would make the comparison with the scorecard stricter, although with 1,000 loans I do not expect a large change.
- **Data pipeline.** The system-level data are collected by hand. With API access, a script could download the ECB and Bundesbank series directly and update the charts on a schedule.
- **Better data.** Re-running everything on the corrected South German Credit data would remove the known labelling problems.

The paper, the notebook and all the code referenced here are in the project repository.